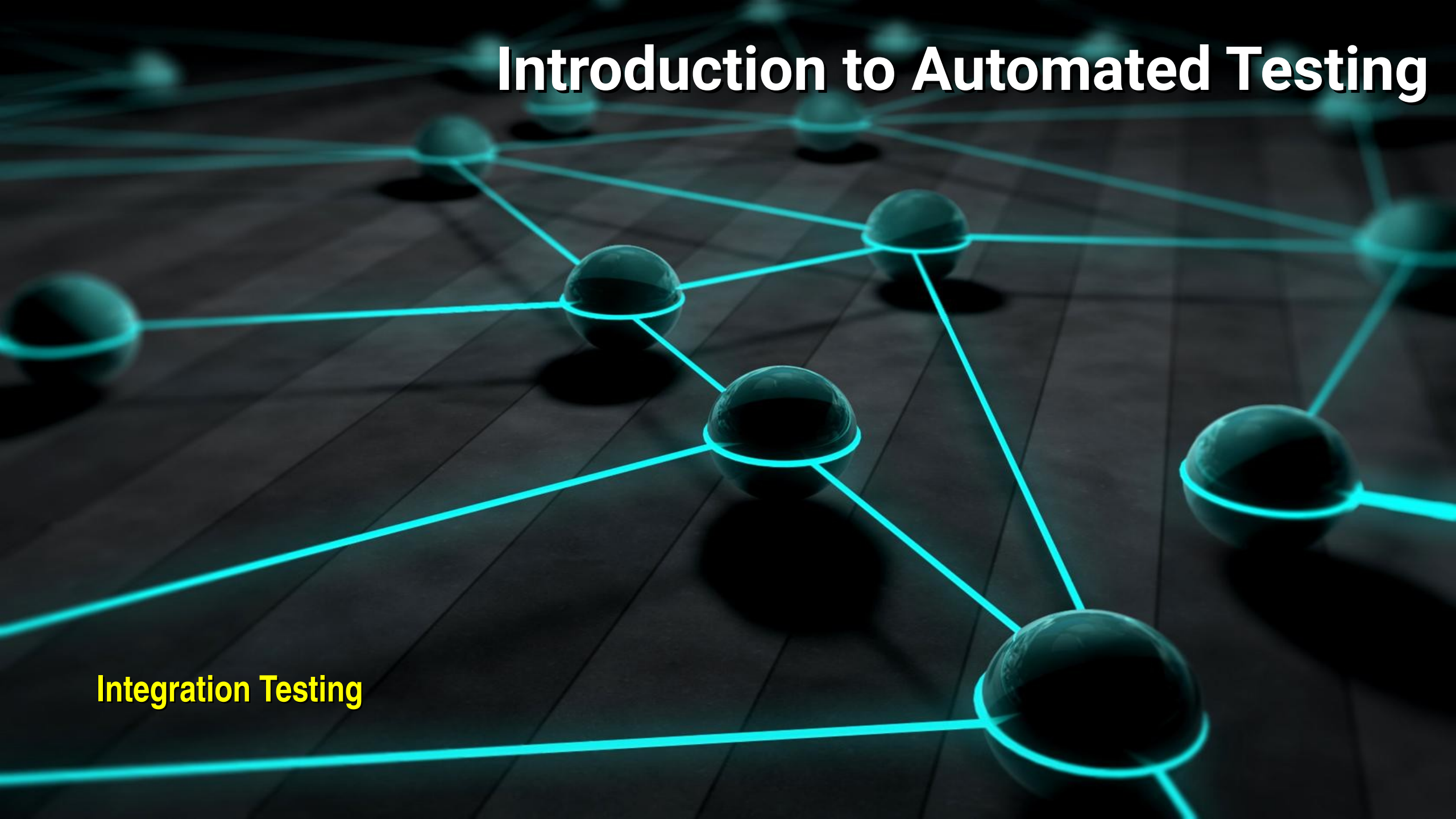


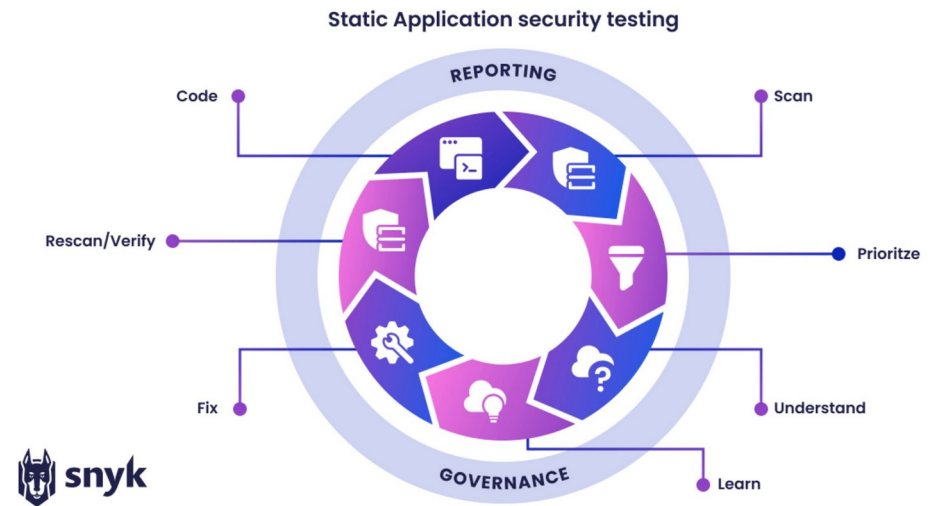
Introduction to Automated Testing

A network diagram consisting of several glowing blue spheres connected by thin, glowing blue lines. The spheres are arranged in a non-uniform pattern across the frame, with some in the foreground and others receding into the background. The lines connect the spheres, creating a web-like structure. The entire scene is set against a dark, textured background that resembles a wooden floor with vertical planks.

Integration Testing

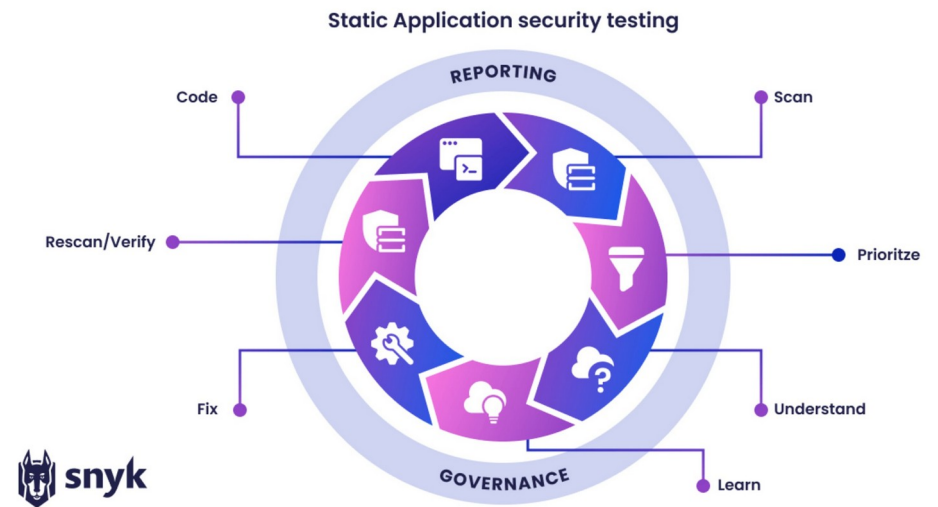
SAST

- Static Application Security Testing
 - Static
 - Analyzes source code or compiled bytecode without executing it
 - Application
 - Focuses on the code your team writes, not the OS or infrastructure
 - Security Testing
 - Looks specifically for security defects, not bugs in general
 - Runs early and often
 - Every commit, every pull request, every CI build
 - The "shift left" philosophy
 - Find vulnerabilities at the cheapest point in the SDLC



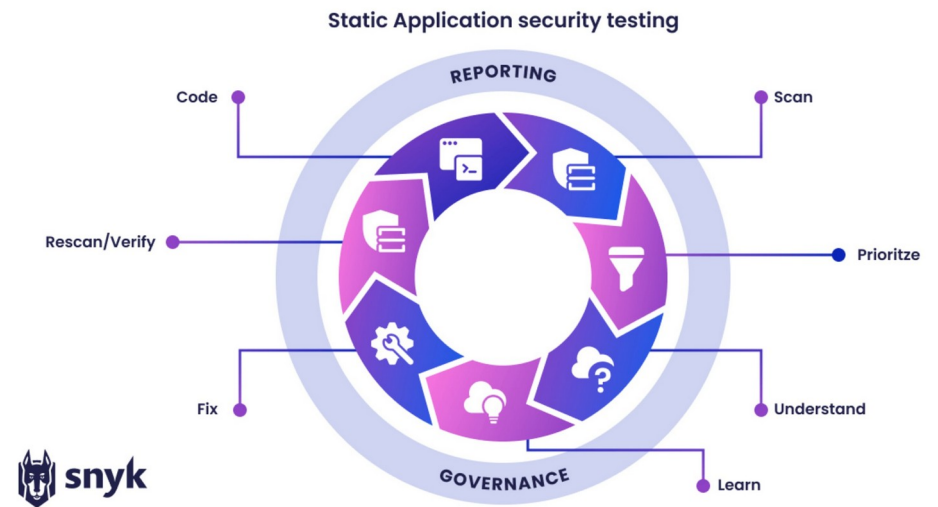
SAST

- Static
 - Code is analyzed without being run
 - The analyzer reads the source files
 - Builds a model of what the code does
 - Looks for patterns that indicate vulnerabilities
 - Compare with dynamic analysis (DAST), which runs the application and probes it from outside
 - Static is faster, cheaper, and earlier in the SDLC but dynamic catches things static cannot
- Application
 - For application code
 - Not for operating system vulnerabilities, network misconfigurations, or hardware issues
 - Those are covered by other tools



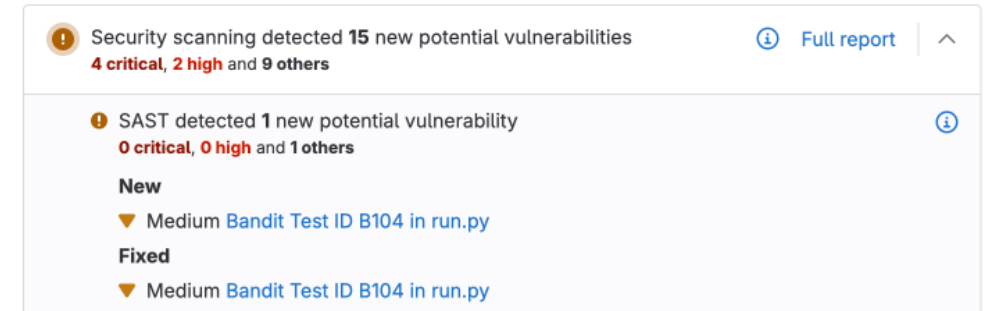
SAST

- Security testing focus
 - SAST tools are tuned to find security defects
 - SQL injection, XSS, hardcoded credentials, weak crypto
 - Some tools also report code quality issues alongside security issues
 - SonarQube is a SAST tool that
 - Does security scans
 - Identifies potential bugs
 - Identifies code smells
 - Provides metrics like cyclomatic complexity
 - Checkmarx is used by enterprises
 - More thorough but requires a licence



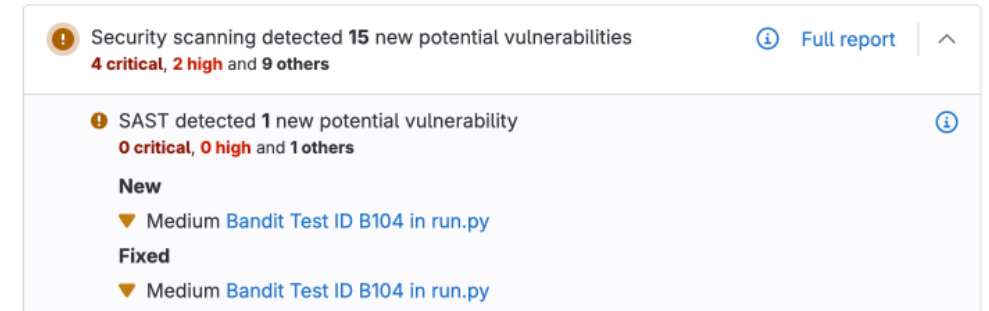
SAST

- How SAST works
 - Step 1: Parse
 - Read source code, build an Abstract Syntax Tree (AST)
 - Step 2: Model
 - Build a control flow graph showing how execution moves through the code
 - Step 3: Analyze
 - Perform taint analysis, tracking untrusted data from sources to sinks
 - Step 4: Report
 - Generate findings for code patterns that match known vulnerability rules



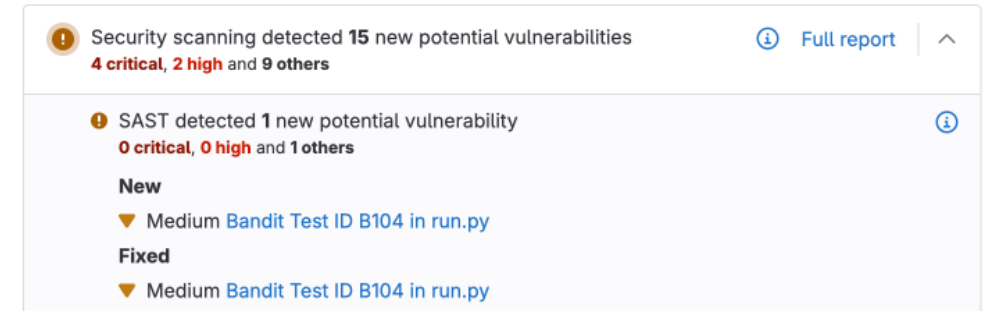
SAST

- What SAST catches
 - Injection vulnerabilities
 - SQL injection, command injection, LDAP injection, XPath injection
 - Cross-site scripting (XSS)
 - Unescaped user input in HTML output
 - Hardcoded credentials
 - Passwords, API keys, tokens in source files
 - Weak cryptography
 - MD5/SHA-1 for security purposes, ECB mode, hardcoded keys, weak random number generatio



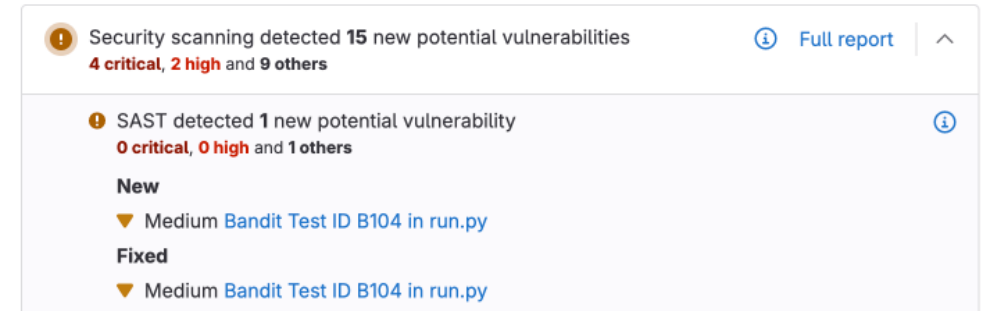
SAST

- What SAST catches
 - Path traversal
 - User input controlling file paths
 - Insecure deserialization
 - Untrusted data passed to readObject(), Jackson, etc.
 - Information leakage
 - Logging sensitive data, exposing stack traces, debug info in responses



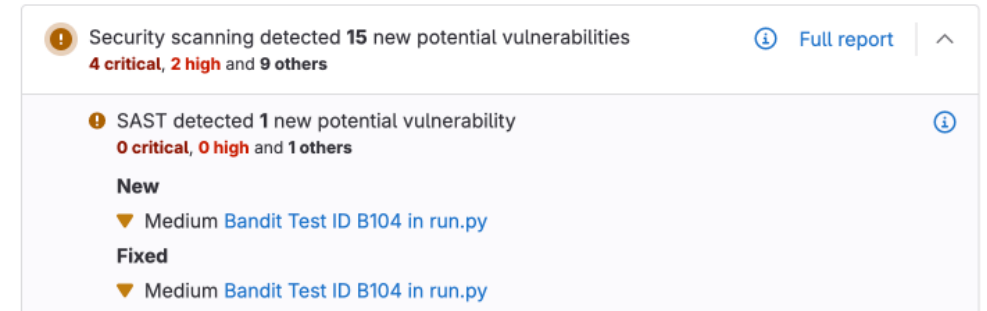
SAST

- Injection
 - The canonical example
 - The pattern is always source-flows-to-sink-without-sanitizer
 - SAST tools detect it well
 - These all share the pattern
 - SQL injection (CWE-89)
 - Command injection (CWE-78)
 - LDAP injection (CWE-90)
 - XPath injection (CWE-91)
 - Header injection (CWE-93)



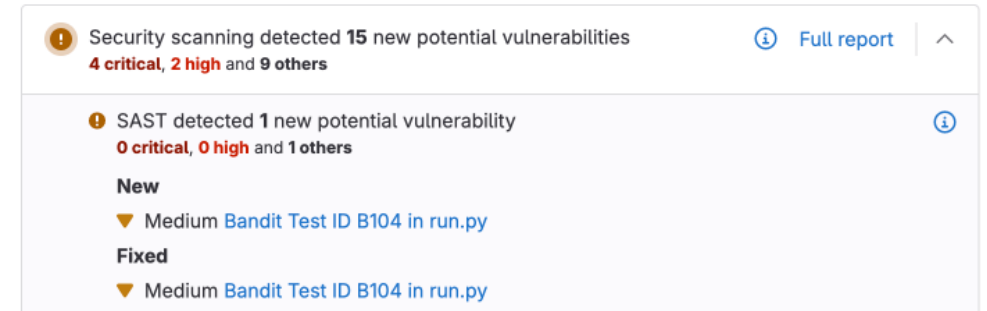
SAST

- Hardcoded credentials
 - Detected by pattern matching
 - Looking for string literals that look like passwords or keys, often with regex against common variable names
 - Less sophisticated than taint analysis but quite effective
- Path traversal
 - Taint analysis applied to file system operations
 - User input flows to new File(...) or Path.of(...)
 - The analyzer asks whether the input was sanitized before use



SAST

- What SAST misses
 - Cryptographic protocol misuse
 - Using crypto correctly per API but wrong for the threat model
 - Information disclosure through observable behavior
 - Timing attacks, error message variation
 - Anything requiring runtime context
 - Environment variables, deployed configuration, user roles in production



CWE

- Common Weakness Enumeration (CWE)
 - CWE is a community-maintained taxonomy of software weaknesses
 - Every common vulnerability class has a CWE number
 - CWE-89 SQL Injection, CWE-79 XSS, CWE-798 Hardcoded Credentials, CWE-22 Path Traversal
 - Every modern SAST tool tags findings with CWE numbers
 - A finding tagged "CWE-89" means the same thing in Checkmarx, SonarQube, Snyk, Semgrep
 - The OWASP top 10 maps to CWE categories
 - The Top 10 is a curated list
 - The CWE is the vocabulary

CWE top 25 most dangerous software weaknesses

#1 CWE-787 Out-of-bounds Write	#2 CWE-79 Cross-site Scripting	#3 CWE-89 SQL Injection	#4 CWE-416 Use After Free	#5 CWE-78 OS Command Injection
#6 CWE-20 Improper Input Validation	#7 CWE-125 Out-of-Bounds Read	#8 CWE-22 Path Traversal	#9 CWE-352 Cross-Site Request Forgery	#10 CWE-434 Unrestricted Upload of File
#11 CWE-862 Missing Authorization	#12 CWE-476 NULL Pointer Dereference	#13 CWE-287 Improper Authentication	#14 CWE-190 Integer Overflow or Wraparound	#15 CWE-502 Deserialization of Untrusted Data
#16 CWE-77 Command Injection	#17 CWE-119 Improper Restriction of Operations...	#18 CWE-798 Use of Hard-coded Credentials	#19 CWE-918 Server-Side Request Forgery	#20 CWE-306 Missing Authentication for Critical...
#21 CWE-362 Race Condition	#22 CWE-269 Improper Privilege Management	#23 CWE-94 Code Injection	#24 CWE-863 Incorrect Authorization	#25 CWE-276 Incorrect Default Permissions



False Positives

- False positives
 - Every SAST tool reports things that aren't necessarily vulnerabilities
 - Because
 - A SAST tool that finds nothing is broken
 - A SAST tool that finds everything is useless
 - The tool errs on the side of reporting, leaves judgment to humans
 - "False positive" means the tool was wrong about the data flow or the risk

CWE top 25 most dangerous software weaknesses				
#1 CWE-787 Out-of-bounds Write	#2 CWE-79 Cross-site Scripting	#3 CWE-89 SQL Injection	#4 CWE-416 Use After Free	#5 CWE-78 OS Command Injection
#6 CWE-20 Improper Input Validation	#7 CWE-125 Out-of-Bounds Read	#8 CWE-22 Path Traversal	#9 CWE-352 Cross-Site Request Forgery	#10 CWE-434 Unrestricted Upload of File
#11 CWE-862 Missing Authorization	#12 CWE-476 NULL Pointer Dereference	#13 CWE-287 Improper Authentication	#14 CWE-190 Integer Overflow or Wraparound	#15 CWE-502 Deserialization of Untrusted Data
#16 CWE-77 Command Injection	#17 CWE-119 Improper Restriction of Operations...	#18 CWE-798 Use of Hard-coded Credentials	#19 CWE-918 Server-Side Request Forgery	#20 CWE-306 Missing Authentication for Critical...
#21 CWE-362 Race Condition	#22 CWE-269 Improper Privilege Management	#23 CWE-94 Code Injection	#24 CWE-863 Incorrect Authorization	#25 CWE-276 Incorrect Default Permissions



False Positives

- False positives
 - Every SAST tool produces them
 - This is a deliberate design choice
 - The tool defers judgment to a human (you) for the cases where it can't be certain
 - The skill is distinguishing real findings from false positives
 - Two general mistake types:
 - Accept everything: treat every finding as real, fix or suppress all of them, burn out on the volume
 - Dismiss everything: assume the tool doesn't understand and dismiss findings without analysis, miss real issues

CWE top 25 most dangerous software weaknesses

#1 CWE-787 Out-of-bounds Write	#2 CWE-79 Cross-site Scripting	#3 CWE-89 SQL Injection	#4 CWE-416 Use After Free	#5 CWE-78 OS Command Injection
#6 CWE-20 Improper Input Validation	#7 CWE-125 Out-of-Bounds Read	#8 CWE-22 Path Traversal	#9 CWE-352 Cross-Site Request Forgery	#10 CWE-434 Unrestricted Upload of File
#11 CWE-862 Missing Authorization	#12 CWE-476 NULL Pointer Dereference	#13 CWE-287 Improper Authentication	#14 CWE-190 Integer Overflow or Wraparound	#15 CWE-502 Deserialization of Untrusted Data
#16 CWE-77 Command Injection	#17 CWE-119 Improper Restriction of Operations...	#18 CWE-798 Use of Hard-coded Credentials	#19 CWE-918 Server-Side Request Forgery	#20 CWE-306 Missing Authentication for Critical...
#21 CWE-362 Race Condition	#22 CWE-269 Improper Privilege Management	#23 CWE-94 Code Injection	#24 CWE-863 Incorrect Authorization	#25 CWE-276 Incorrect Default Permissions



False Positives

- False positives
 - Every SAST tool produces them
 - This is a deliberate design choice
 - The tool defers judgment to a human (you) for the cases where it can't be certain
 - The skill is distinguishing real findings from false positives
 - Two general mistake types:
 - Accept everything: treat every finding as real, fix or suppress all of them, burn out on the volume
 - Dismiss everything: assume the tool doesn't understand and dismiss findings without analysis, miss real issues

CWE top 25 most dangerous software weaknesses

#1 CWE-787 Out-of-bounds Write	#2 CWE-79 Cross-site Scripting	#3 CWE-89 SQL Injection	#4 CWE-416 Use After Free	#5 CWE-78 OS Command Injection
#6 CWE-20 Improper Input Validation	#7 CWE-125 Out-of-Bounds Read	#8 CWE-22 Path Traversal	#9 CWE-352 Cross-Site Request Forgery	#10 CWE-434 Unrestricted Upload of File
#11 CWE-862 Missing Authorization	#12 CWE-476 NULL Pointer Dereference	#13 CWE-287 Improper Authentication	#14 CWE-190 Integer Overflow or Wraparound	#15 CWE-502 Deserialization of Untrusted Data
#16 CWE-77 Command Injection	#17 CWE-119 Improper Restriction of Operations...	#18 CWE-798 Use of Hard-coded Credentials	#19 CWE-918 Server-Side Request Forgery	#20 CWE-306 Missing Authentication for Critical...
#21 CWE-362 Race Condition	#22 CWE-269 Improper Privilege Management	#23 CWE-94 Code Injection	#24 CWE-863 Incorrect Authorization	#25 CWE-276 Incorrect Default Permissions



Triage Workflow

- Confirm
 - Read the data flow trace the tool provides
 - Most SAST tools show the path from source to sink, line by line.
 - Walk through it mentally
 - Is the data actually untrusted?
 - Does it actually reach the sink?
 - Is there a sanitizer the tool missed?
 - At the end of this step, you have a position
 - Yes, this is real
 - Or no, the tool got something wrong.s

For each finding:

1. CONFIRM -- Read the data flow. Does the alleged vulnerability exist?
2. CLASSIFY -- Pick one:
 - True positive, fix now
 - True positive, accepted risk (document)
 - False positive (document why)
 - Out of scope (test code, generated, etc.)
3. ACT -- Fix the code OR document the suppression OR confirm scope
4. VERIFY -- Re-scan to confirm fix or that suppression applied correctly



Triage Workflow

- Classify
 - Four categories cover almost every finding
 - True positive, fix now
 - The vulnerability is real, has no compensating control, should be remediated immediately
 - SQL injection in user-facing code is the canonical example.
 - True positive, accepted risk
 - The vulnerability is real but mitigated by other factors
 - The endpoint is internal-only and behind authentication
 - The system is air-gapped
 - The compensating control is documented and reviewed by security
 - The classification acknowledges the issue while explaining why immediate fix isn't required.

For each finding:

1. CONFIRM -- Read the data flow. Does the alleged vulnerability exist?
2. CLASSIFY -- Pick one:
 - True positive, fix now
 - True positive, accepted risk (document)
 - False positive (document why)
 - Out of scope (test code, generated, etc.)
3. ACT -- Fix the code OR document the suppression OR confirm scope
4. VERIFY -- Re-scan to confirm fix or that suppression applied correctly



Triage Workflow

- Classify
 - False positive: the tool is wrong
 - Either the data flow doesn't actually exist (sanitizer missed)
 - Or the alleged risk doesn't apply (the sink isn't actually sensitive)
 - Document why so a future developer or auditor needs to understand the reasoning
 - Out of scope:
 - The finding is in code that doesn't get deployed
 - Test code, generated code, third-party libraries
 - Often these can be excluded by configuration before scanning, but if they sneak in, classify them as out of scope with a note.

For each finding:

1. CONFIRM -- Read the data flow. Does the alleged vulnerability exist?
2. CLASSIFY -- Pick one:
 - True positive, fix now
 - True positive, accepted risk (document)
 - False positive (document why)
 - Out of scope (test code, generated, etc.)
3. ACT -- Fix the code OR document the suppression OR confirm scope
4. VERIFY -- Re-scan to confirm fix or that suppression applied correctly



Triage Workflow

- Act
 - For "fix now," change the code
 - For "accepted risk" and "false positive," create the suppression in the tool with the documented justification
 - For "out of scope," either fix the scan configuration or mark them in the tool.
- Verify
 - Always re-scan after fixing
 - The finding should be gone (or at least transformed into a different finding).
 - If it's still there, your fix didn't address what the tool was complaining about.

For each finding:

1. CONFIRM -- Read the data flow. Does the alleged vulnerability exist?
2. CLASSIFY -- Pick one:
 - True positive, fix now
 - True positive, accepted risk (document)
 - False positive (document why)
 - Out of scope (test code, generated, etc.)
3. ACT -- Fix the code OR document the suppression OR confirm scope
4. VERIFY -- Re-scan to confirm fix or that suppression applied correctly



Example

- Triage
 - Read the trace
 - Does the user-supplied accountId actually reach the SQL execution unsanitized?
 - Inspect the code.
 - If the query uses + concatenation: it's real
 - If it uses ? placeholders with setString: false positive, then the tool missed the parameterization
 - Decide
 - True positive, fix now
 - False positive, suppress with justification.
 - Re-scan
 - Confirm the finding is gone after the fix

[HIGH] CWE-89 SQL Injection in TransactionRepository.java:42

Source: request.getParameter("accountId") at TransactionController.java:18
Path: accountId → transactionService.getByAccount(accountId) → ...
Sink: statement.executeQuery(query) at TransactionRepository.java:42

Suggested fix: Use parameterized queries.



Severity Levels and Prioritization

- Not all findings are equal
 - Critical / High
 - Direct exploitability, business impact
 - RCE, SQL injection, hardcoded prod credentials
 - Medium
 - Exploitable in some contexts
 - XSS in admin pages, weak crypto for non-secret data
 - Low
 - Defense-in-depth issues
 - Verbose error messages, deprecated APIs, missing security headers
 - Info
 - Code quality observations, not exploitable on their own
 - CI pipeline typically blocks merge on Critical/High; tracks Medium; informational for Low

[HIGH] CWE-89 SQL Injection in TransactionRepository.java:42

Source: request.getParameter("accountId") at TransactionController.java:18
Path: accountId → transactionService.getByAccount(accountId) → ...
Sink: statement.executeQuery(query) at TransactionRepository.java:42

Suggested fix: Use parameterized queries.



Suppression Justifications

- Writing Justifications for audit
 - Bad:
 - "False positive."
 - "I don't think this is a problem."
 - Better:
 - "False positive. The tool identified accountId flowing to executeQuery without sanitization, but inspection shows the JDBC PreparedStatement uses positional parameters (line 42, statement.setString(1, accountId))."
 - The tool's data flow analysis did not recognize the parameterization through the prepareStatement / setString idiom. No remediation required."
 - Best:
 - [the better text above] +
 - "Reviewed with [security team contact] on [date]. Filed as suppression #1234 with reference to internal AppSec wiki page <https://...>"

Concept	SonarQube (lab)	Checkmarx (likely at the bank)
Severity	Blocker / Critical / Major / Minor / Info	Critical / High / Medium / Low / Information
Finding identity	Issue with key, rule, severity, location	Result with query name, severity, CWE
Triage status	Open / Confirmed / Resolved / Reopened	To Verify / Confirmed / Not Exploitable / Proposed Not Exploitable
False positive flag	Resolution: "False Positive" with comment	Mark as "Not Exploitable" with justification
Vulnerability vocab	CWE numbers	CWE numbers (same)
Scan trigger	Maven/Gradle plugin or CLI	CI integration, IDE plugin, CLI



Suppression Justifications

- A bad justification ("false positive")
 - Tells the future reader nothing.
 - They have to redo your analysis from scratch, often without the context you had
 - They may incorrectly accept your dismissal
 - They may incorrectly reject it
 - Either way, you've left them worse off than necessary.

Concept	SonarQube (lab)	Checkmarx (likely at the bank)
Severity	Blocker / Critical / Major / Minor / Info	Critical / High / Medium / Low / Information
Finding identity	Issue with key, rule, severity, location	Result with query name, severity, CWE
Triage status	Open / Confirmed / Resolved / Reopened	To Verify / Confirmed / Not Exploitable / Proposed Not Exploitable
False positive flag	Resolution: "False Positive" with comment	Mark as "Not Exploitable" with justification
Vulnerability vocab	CWE numbers	CWE numbers (same)
Scan trigger	Maven/Gradle plugin or CLI	CI integration, IDE plugin, CLI



Suppression Justifications

- A good justification has three properties:
 - States the tool's allegation
 - "The tool identified X flowing to Y."
 - This shows you understood what the tool was reporting, not just that you disagreed.
 - Explains the actual code behavior
 - "Inspection shows the code uses parameterized queries via PreparedStatement.setString."
 - This is the technical fact that contradicts the tool's allegation.
 - Identifies WHY the tool was wrong
 - "The tool's data flow analysis did not recognize the JDBC parameterization idiom."
 - This is the diagnostic insight that explains the false positive at the level of the tool's behavior, not just the code.

Concept	SonarQube (lab)	Checkmarx (likely at the bank)
Severity	Blocker / Critical / Major / Minor / Info	Critical / High / Medium / Low / Information
Finding identity	Issue with key, rule, severity, location	Result with query name, severity, CWE
Triage status	Open / Confirmed / Resolved / Reopened	To Verify / Confirmed / Not Exploitable / Proposed Not Exploitable
False positive flag	Resolution: "False Positive" with comment	Mark as "Not Exploitable" with justification
Vulnerability vocab	CWE numbers	CWE numbers (same)
Scan trigger	Maven/Gradle plugin or CLI	CI integration, IDE plugin, CLI

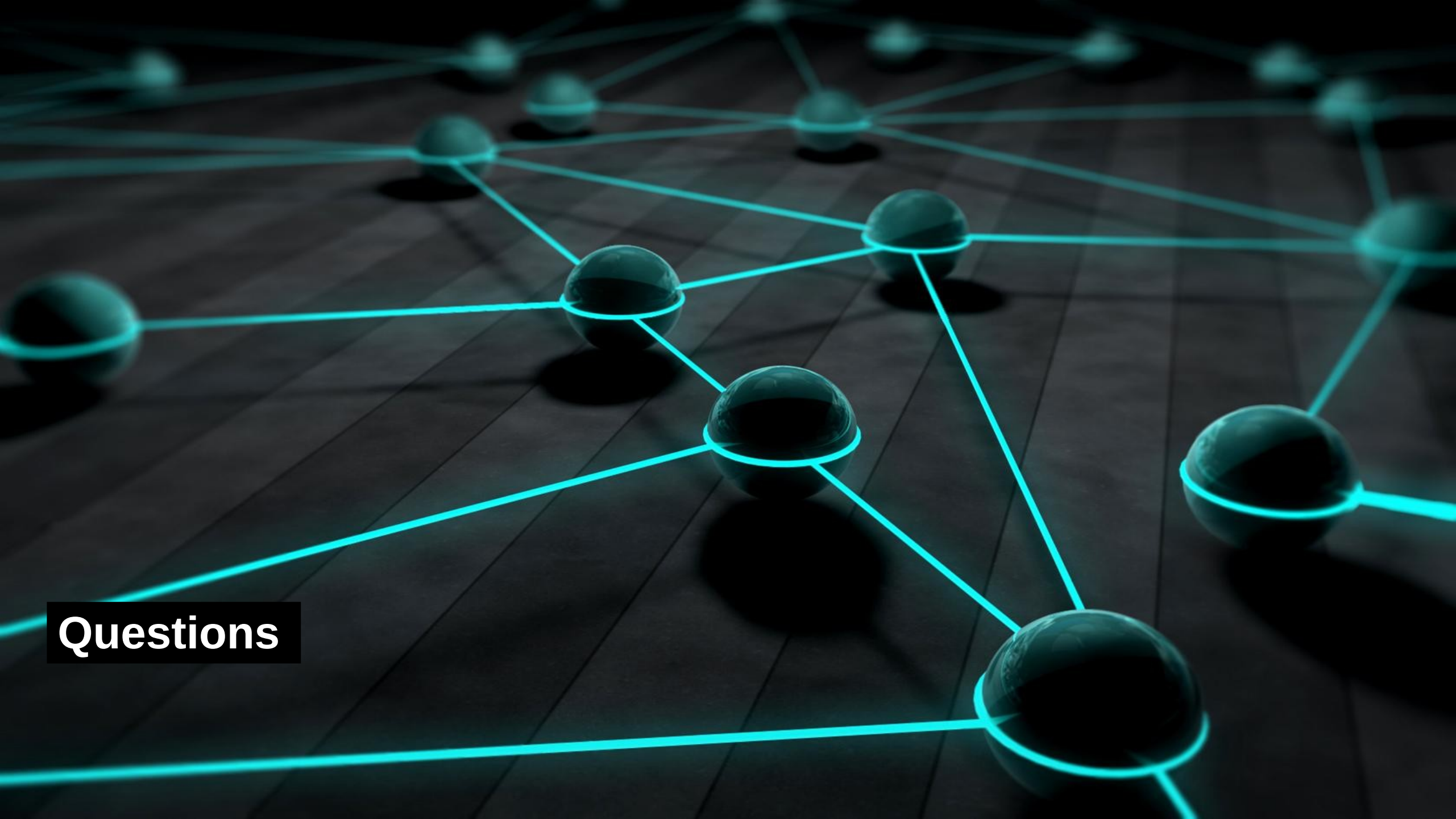


Suppression Justifications

- The "Best" version
 - Adds review and traceability
 - A senior security person reviewed and concurred
 - A unique suppression ID in the tracking system links to the conversation
 - A wiki link or document reference provides extended context
 - This level of rigor is appropriate for high-severity findings or for codebases under regulatory scrutiny

Concept	SonarQube (lab)	Checkmarx (likely at the bank)
Severity	Blocker / Critical / Major / Minor / Info	Critical / High / Medium / Low / Information
Finding identity	Issue with key, rule, severity, location	Result with query name, severity, CWE
Triage status	Open / Confirmed / Resolved / Reopened	To Verify / Confirmed / Not Exploitable / Proposed Not Exploitable
False positive flag	Resolution: "False Positive" with comment	Mark as "Not Exploitable" with justification
Vulnerability vocab	CWE numbers	CWE numbers (same)
Scan trigger	Maven/Gradle plugin or CLI	CI integration, IDE plugin, CLI





Questions